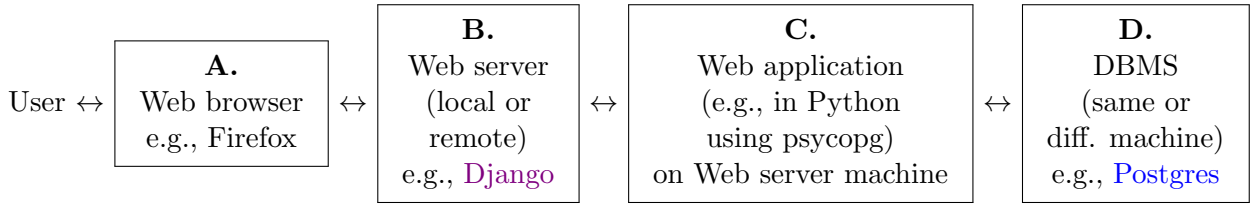
CSC_51053_EP, Lab Session: Application Programming on top of a Database Management System (PostgreSQL)

Ioana Manolescu, Madhulika Mohanty, Garima Gaur

14 November 2025

Overview

Databases are often exploited via **Web applications**, following the schema below:



In this lab, you will first use the DBMS from a Python program (**C.** ↔ **D.** above) (Sec. 1). Then, you will install a Web server, deploy a simple application, and use it locally (User ↔ **A.** ↔ **B.** ↔ **C.** above), in Sec. 2. Finally, you will put them all together into an application instantiating all the elements above.

1 Accessing Postgres from python using `psycopg`

Make sure a PostgreSQL server is running on your machine.

Create a local directory for this lab and copy the file psycatalog.py in this directory. Modify the values of `dbame`, `user`, `password` and `port` as appropriate to enable connection to your server.

Install the `psycopg` library which provides an API for accessing an RDBMS from python[1].

Running `psycatalog.py` should show you a list of the tables in the database. Note that the query uses Postgres' built-in pg_catalog.pg_tables table.

More information about the Postgres catalog is at: https://www.postgresql.org/docs/current/catalogs.html.

2 Setting up a Web server and application with `Django`

Open (online, successively) these files from Moodle, and follow the instructions therein (text files to facilitate copy-paste):

- **Installing and launching a web server – basic application:** django-1.txt
- **Formatting the content of webpages as HTML:** django-2.txt
- **Taking user input from a webpage and returning results from back-end:** django-3.txt

[1] Python package installation notes; Psycopg documentation

3 The IMDB database

Next, you will develop and deploy a Django Web application based on the movie database (a snapshot of the Internet Movie Database, or IMDB for short) from the previous lab. If you do not have it on your system, load it using the instructions from the previous lab.

Inspect the tables in this database. Figure 1 shows the schema of this database, generated using DBSchema[2] application. Observe the primary keys and the foreign keys which will be useful in joining the tables.

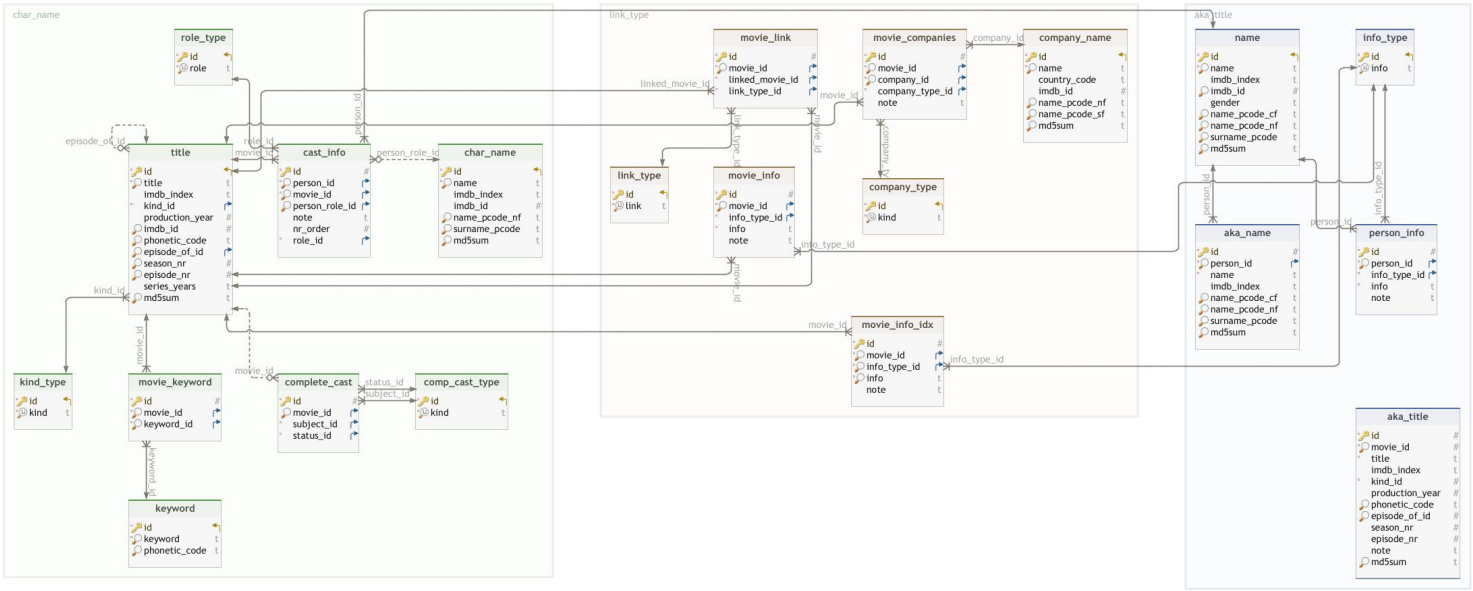


Figure 1: IMDb database schema

4 IMDB Web application

Using Django as in Section 2, build two Web pages leveraging the IMDB database.

1. Create a page available at `http://127.0.0.1:8000/welcome` showing the movies from the current year (2024).
2. Create an interactive page available at `http://127.0.0.1:8000/searchartist` which asks for the name of an artist, and displays their 10 most recent movies. If more than one artist match the input name, display 10 most recent movies for each of them.

5 Enforcing Access Control

Create a new database user "viewer": CREATE USER viewer. Create a database view "welcomeview" using the query used for item 1 above. Grant the new user rights to perform SELECT on this view: GRANT SELECT ON welcomeview TO viewer;

1. Modify the welcome page to use this view to render the results while connecting as "viewer". Do you see the result on the website?
2. Perform the username change on the "searchartist" page also. Do you see the results?

Advanced: Perform exception handling in Python to catch the errors and display a nicer output for the user instead of a Django error page.

[2]https://dbschema.com/